

Evidencia de producto: GA8-220501096-AA1-EV02 módulos
integrados

Aprendiz:

Víctor Darío Morales López

Análisis y desarrollo de software

Ficha :3118301

Instructor (a):

Doris Ordoñez

2026

TABLA DE CONTENIDO

INTRODUCCION	3
OBJETIVOS.....	4
DESCRIPCIÓN DEL PROYECTO.....	5
HERRAMIENTAS UTILIZADAS.....	5
ESTRUCTURA DEL PROYECTO	6
FRONTEND FUNCIONANDO	7
BACKEND FUNCIONANDO	8
INTEGRACIÓN DE MÓDULOS.....	9
MÓDULO SERVICIOS	9
Pruebas al modulo servicios con postman	9
MÓDULO CLIENTES.....	12
Pruebas al módulo clientes con postman	12
MÓDULO PROYECTOS	15
Pruebas al módulo proyectos con postman.....	15
MÓDULO USUARIOS	17
Pruebas al módulo usuarios con postman.....	18
CONCLUSIÓN.....	21

INTRODUCCION

El presente documento describe la integración completa de los módulos desarrollados para la aplicación web Muvic DigiMark. En esta fase se realizó la conexión entre el frontend desarrollado en React, el backend desarrollado con Node.js y Express, y la base de datos MySQL, permitiendo el funcionamiento integral del sistema. Además, se documentan las pruebas realizadas, la configuración del ambiente de desarrollo y los resultados obtenidos durante la validación de la aplicación.

OBJETIVOS

Objetivo General

Integrar los módulos desarrollados para la aplicación web Muvic DigiMark garantizando la comunicación entre la interfaz gráfica, la lógica de negocio y la base de datos.

Objetivos Específicos

- Integrar frontend y backend.
- Configurar la base de datos.
- Validar los servicios mediante pruebas.
- Documentar el entorno de desarrollo.
- Gestionar el código mediante GitHub.

DESCRIPCIÓN DEL PROYECTO

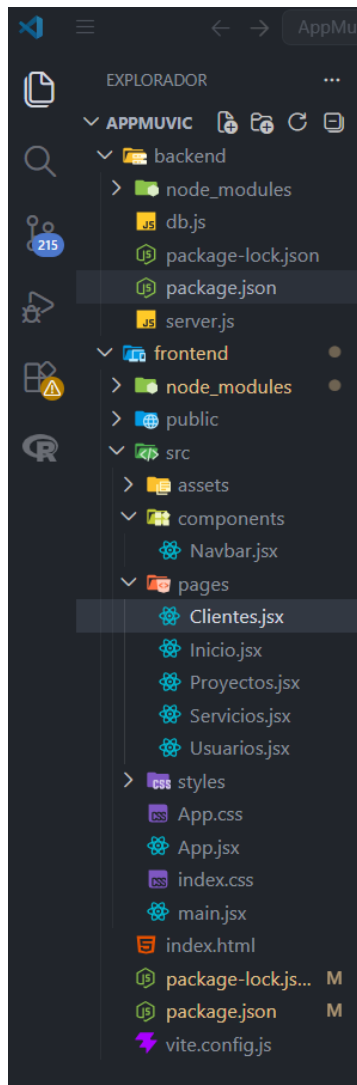
Muvic DigiMark es una plataforma web diseñada para la administración de servicios de marketing digital y desarrollo de software. La aplicación permite gestionar clientes, proyectos, servicios y usuarios mediante una interfaz intuitiva conectada a una base de datos relacional.

HERRAMIENTAS UTILIZADAS

Tecnología	Función
React	Frontend
Vite	Entorno de desarrollo
Node.js	Backend
Express	API REST
MySQL	Base de datos
Postman	Pruebas
GitHub	Control de versiones
VS Code	IDE

ESTRUCTURA DEL PROYECTO

Estructura de carpetas del proyecto Muvic DigiMark



La aplicación fue organizada siguiendo una arquitectura basada en componentes utilizando React. La carpeta *pages* contiene los módulos principales del sistema (Inicio, Servicios, Clientes, Proyectos y Usuarios), mientras que la carpeta *components* almacena componentes reutilizables y *assets* contiene los recursos gráficos utilizados por la aplicación.

FRONTEND FUNCIONANDO

```
PS C:\AppMuvic> cd frontend
PS C:\AppMuvic\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.1.0 ready in 150 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



La figura muestra la interfaz principal de Muvic DigiMark ejecutándose correctamente desde el servidor de desarrollo Vite. Se evidencian los módulos de Usuarios, Clientes, Servicios y Proyectos integrados dentro del sistema.

BACKEND FUNCIONANDO

```
PS C:\AppMuvic\backend> node server.js
Servidor de Muvic DigiMark corriendo en http://localhost:4000
Conectado exitosamente a la base de datos MySQL.
```

```
localhost:4000/api/clientes

Impresión con formato estilístico ☒

{
  "id": 1,
  "nombre": "Vibel Store",
  "email": "contacto@vibelstore.com"
},
{
  "id": 2,
  "nombre": "Constructora Alfa S.A.",
  "email": "gerencia@constructoraalfa.co"
},
{
  "id": 3,
  "nombre": "Gym Vital Colombia",
  "email": "administracion@gymvital.com"
},
{
  "id": 4,
  "nombre": "Restaurante El Portal",
  "email": "proveedores@elportalrestobar.com"
}
```

Prueba de funcionamiento del backend desarrollada en Node.js y Express. Se verifica la conexión exitosa con la base de datos MySQL y la consulta de información mediante el endpoint /api/clientes, confirmando la correcta comunicación entre la API y la base de datos.

INTEGRACIÓN DE MÓDULOS

MÓDULO SERVICIOS

The screenshot shows a web browser at localhost:5173/servicios. The application header is 'Muvic DigiMark' with navigation links: Inicio, Usuarios, Clientes, **Servicios**, and Proyectos. The main section is titled 'Módulo de Servicios'.

At the top of the module, there is a form to add a new service with two input fields: 'Descripción del Servicio' (containing 'Ej. Diseño Web Responsivo') and 'Precio Unitario (COP)' (containing '500000'). A '+ Guardar Servicio' button is to the right.

Below the form is a table listing existing services:

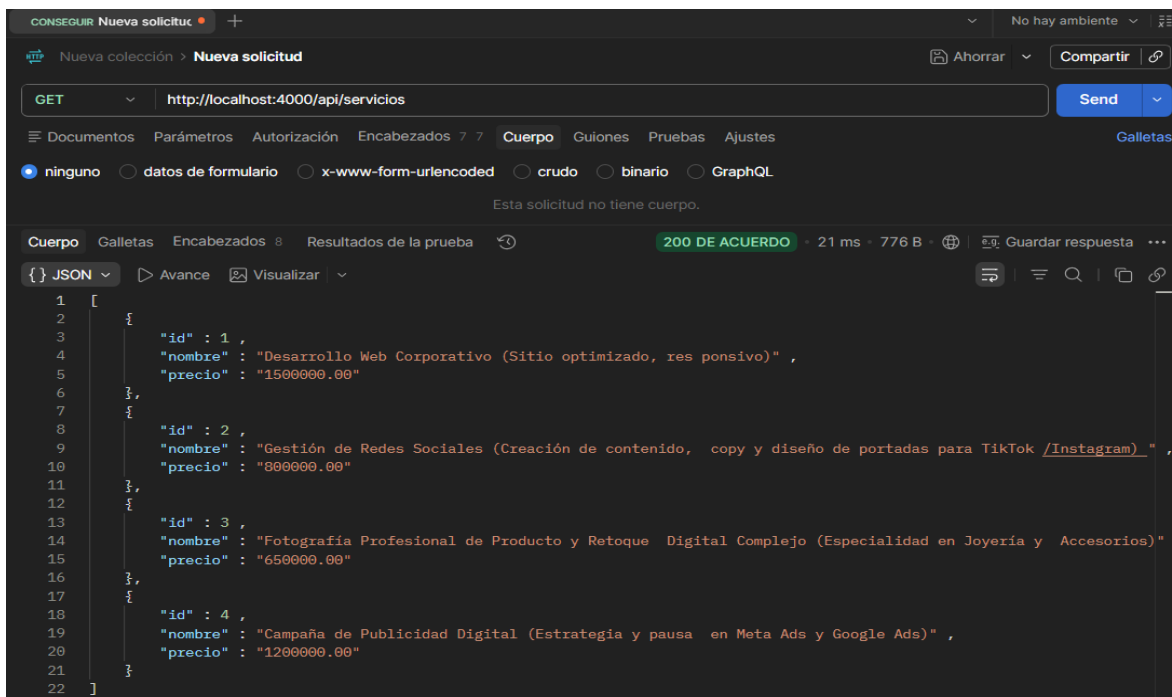
ID	Línea de Servicio	Tarifa Base	Acciones
1	Desarrollo Web Corporativo (Sitio optimizado, responsivo)	\$1.500.000	Editar Eliminar
2	Gestión de Redes Sociales (Creación de contenido, copy y diseño de portadas para TikTok/Instagram)	\$800.000	Editar Eliminar
3	Fotografía Profesional de Producto y Retoque Digital Complejo (Especialidad en Joyería y Accesorios)	\$650.000	Editar Eliminar
4	Campaña de Publicidad Digital (Estrategia y pauta en Meta Ads y Google Ads)	\$1.200.000	Editar Eliminar

Permite registrar, consultar, editar y eliminar servicios mediante conexión directa con la API RESTy almacenamiento persistente en MySQL.

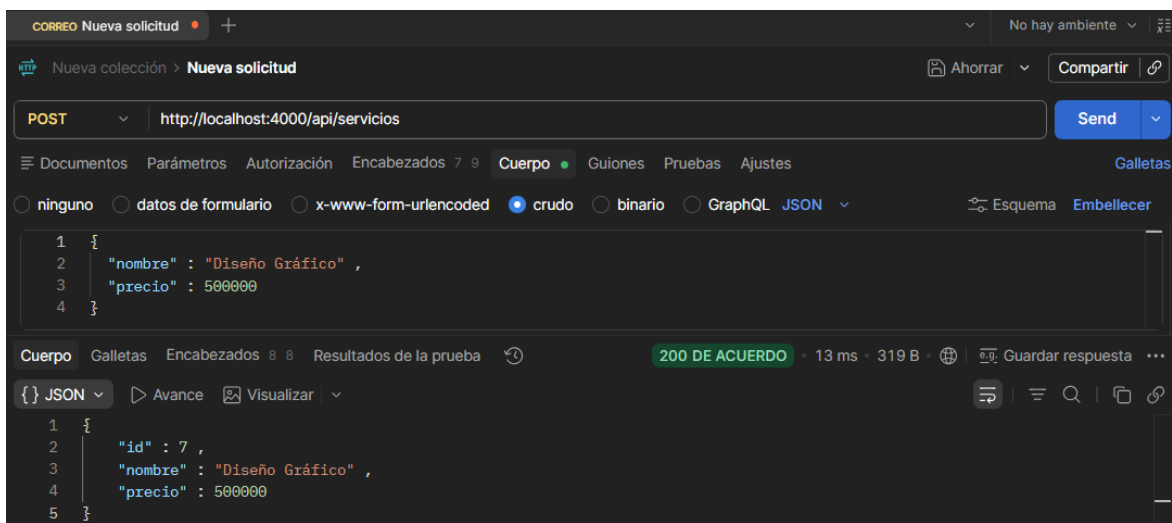
Pruebas al modulo servicios con postman

Prueba 1. Consulta de servicios (GET) URL <http://localhost:4000/api/servicios>

Se realizó una consulta de los servicios registrados mediante una petición GET. La API respondió con un listado de servicios en formato JSON, validando la comunicación entre el backend y la base de datos.



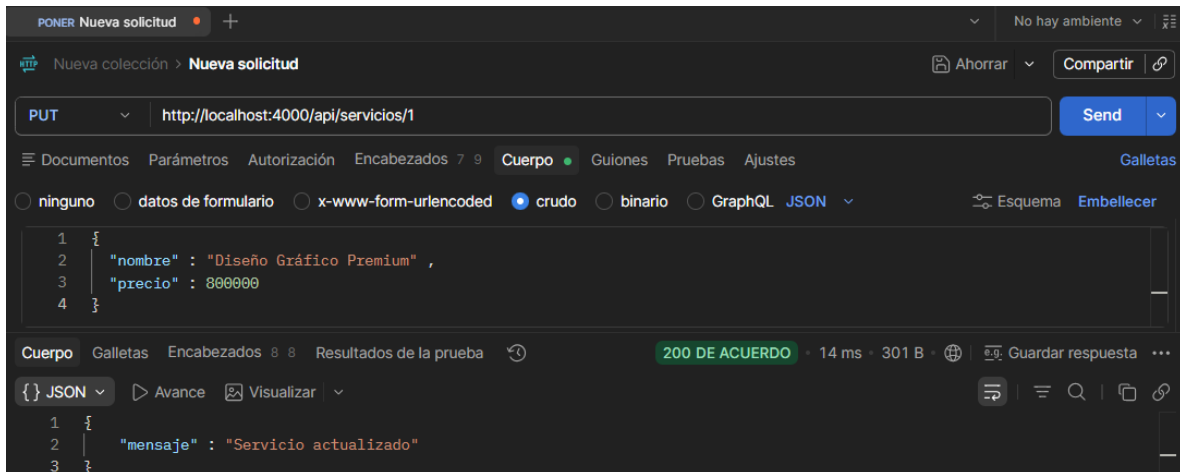
Prueba 2. Registro de servicio (POST) URL <http://localhost:4000/api/servicios>



Se ejecutó una prueba de creación de servicios mediante una petición POST. La información suministrada fue almacenada correctamente en la base de datos, verificando el funcionamiento de la operación de inserción del módulo Servicios.

Prueba 3. Actualización de servicio (PUT):URL

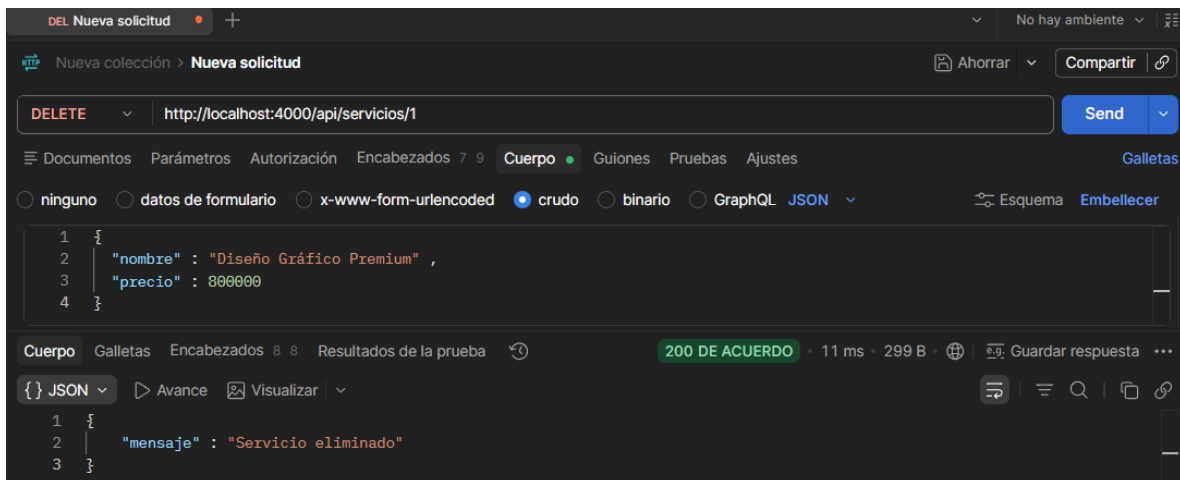
<http://localhost:4000/api/servicios/1>



Se realizó una prueba de actualización de servicios mediante una petición PUT. El sistema modificó correctamente la información del servicio seleccionado y confirmó la actualización de los datos en la base de datos.

Prueba 4. Eliminación de servicio (DELETE): URL

<http://localhost:4000/api/servicios/1>



Se ejecutó una prueba de eliminación mediante una petición DELETE. El sistema eliminó satisfactoriamente el servicio seleccionado, validando el correcto funcionamiento de la operación de borrado.

MÓDULO CLIENTES

Módulo de Clientes

Nombre de Empresa / Cliente: Email de Contacto: + Guardar Cliente

ID	Cliente / Marca	Email Corporativo	Acciones
1	Vibel Store	contacto@vibelstore.com	<button>Editar</button> <button>Eliminar</button>
2	Constructora Alfa S.A.	gerencia@constructoraalfa.co	<button>Editar</button> <button>Eliminar</button>
3	Gym Vital Colombia	administracion@gymvital.com	<button>Editar</button> <button>Eliminar</button>
4	Restaurante El Portal	proveedores@elportalrestobar.com	<button>Editar</button> <button>Eliminar</button>

Permite administrar la información de clientes utilizando operaciones CRUD integradas con la base de datos.

Pruebas al módulo clientes con postman

Prueba 1. Consulta de clientes (GET): URL <http://localhost:4000/api/clientes>

CONSEGUIR Nueva solicitud

Nueva colección > Nueva solicitud

GET http://localhost:4000/api/clientes

Send

Documentos Parámetros Autorización Encabezados 7 7 Cuerpo Gulones Pruebas Ajustes Galletas

ninguno datos de formulario x-www-form-urlencoded crudo binario GraphQL

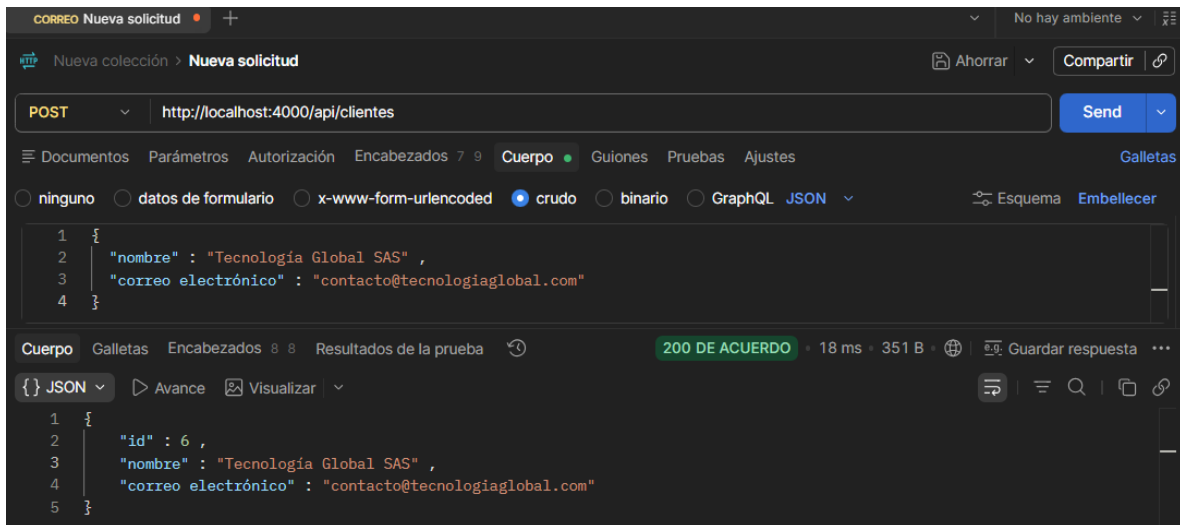
Esta solicitud no tiene cuerpo.

Cuerpo Galletas Encabezados 8 8 Resultados de la prueba 200 DE ACUERDO - 6 ms - 580 B Guardar respuesta

```
{
  "JSON"
}
[
  {
    "id": 1,
    "nombre": "Vibel Store",
    "correo electrónico": "contacto@vibelstore.com"
  },
  {
    "id": 2,
    "nombre": "Constructora Alfa SA",
    "correo electrónico": "gerencia@constructoraalfa.co"
  },
  {
    "id": 3,
    "nombre": "Gimnasio Vital Colombia",
    "correo electrónico": "administracion@gymvital.com"
  },
  {
    "id": 4,
    "nombre": "Restaurante El Portal",
    "correo electrónico": "proveedores@elportalrestobar.com"
  }
]
```

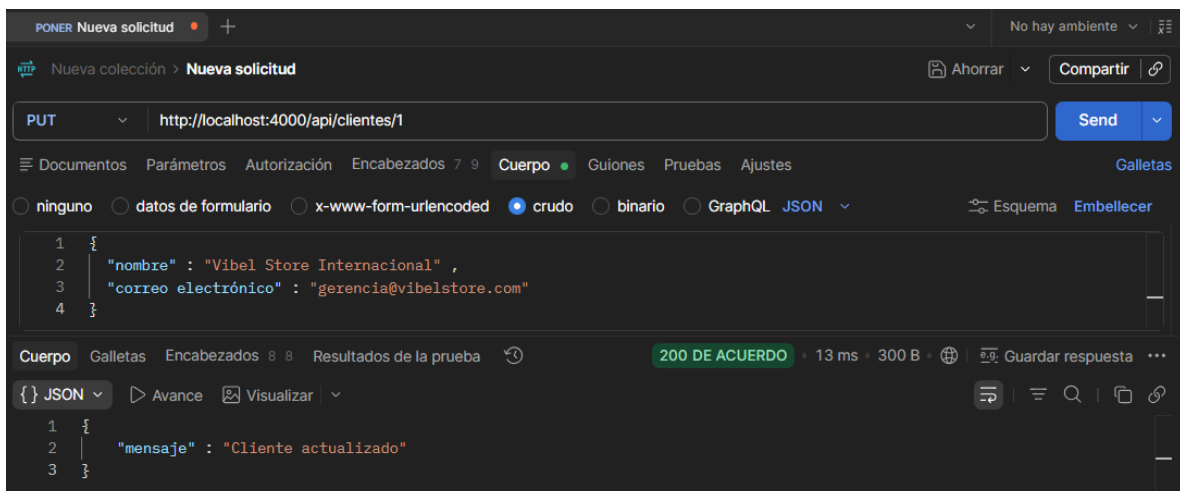
Se realizó una prueba de consulta de clientes mediante una petición GET al endpoint correspondiente. El sistema respondió con código de estado 200 OK y mostró la información almacenada en la base de datos en formato JSON. Esta prueba permitió verificar la correcta comunicación entre la API REST y la base de datos MySQL.

Prueba 2. Registro de cliente (POST): URL <http://localhost:4000/api/clientes>



Se realizó una prueba de inserción de datos mediante una petición POST. El sistema registró correctamente un nuevo cliente en la base de datos y confirmó la operación de almacenamiento de información, validando el funcionamiento del módulo clientes.

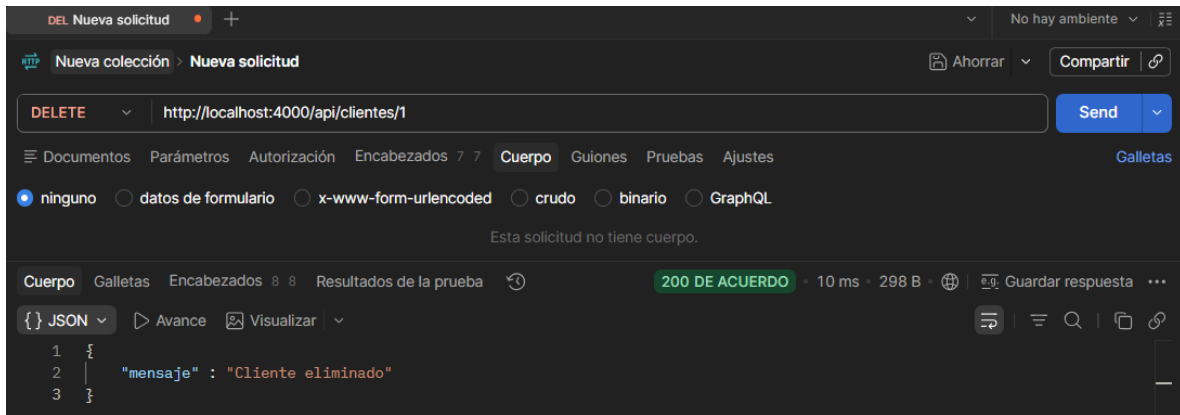
Prueba 3. actualización de cliente (PUT): URL <http://localhost:4000/api/clientes/1>



Se ejecutó una prueba de actualización de información mediante una petición PUT. El sistema modificó correctamente los datos del cliente seleccionado y actualizó el registro correspondiente en la base de datos.

Prueba 4. eliminación de cliente (DELETE): URL

<http://localhost:4000/api/clientes/1>



Se realizó una prueba de eliminación mediante una petición DELETE. El sistema eliminó correctamente el cliente seleccionado y confirmó la operación realizada sobre la base de datos.

MÓDULO PROYECTOS

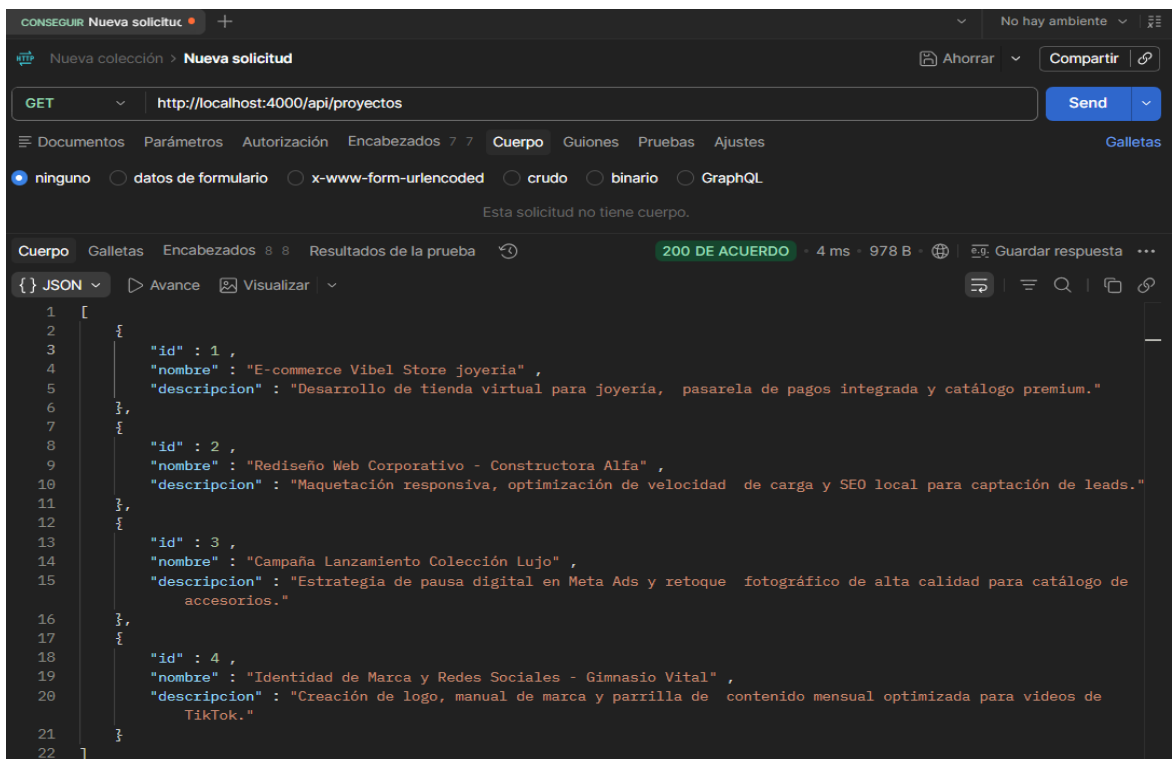


ID	Proyecto	Descripción / Estado	Acciones
1	E-commerce Vibel Store joyeria	Desarrollo de tienda virtual para joyería, pasarela de pagos integrada y catálogo premium.	<button>Editar</button> <button>Eliminar</button>
2	Rediseño Web Corporativo - Constructora Alfa	Maquetación responsiva, optimización de velocidad de carga y SEO local para captación de leads.	<button>Editar</button> <button>Eliminar</button>
3	Campaña Lanzamiento Colección Luxury	Estrategia de pauta digital en Meta Ads y retoque fotográfico de alta calidad para catálogo de accesorios.	<button>Editar</button> <button>Eliminar</button>
4	Identidad de Marca y Social Media - Gym Vital	Creación de logo, manual de marca y parrilla de contenido mensual optimizada para videos de TikTok.	<button>Editar</button> <button>Eliminar</button>

Permite registrar y controlar los proyectos desarrollados para los clientes, almacenando información relevante sobre su seguimiento y estado.

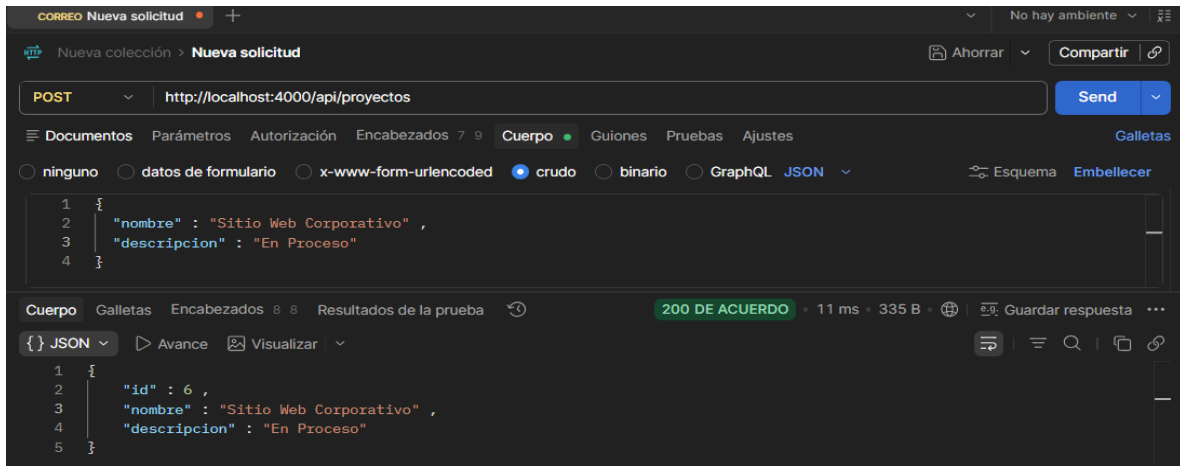
Pruebas al módulo proyectos con postman

Prueba 1. Consulta de proyectos (GET): URL <http://localhost:4000/api/proyectos>



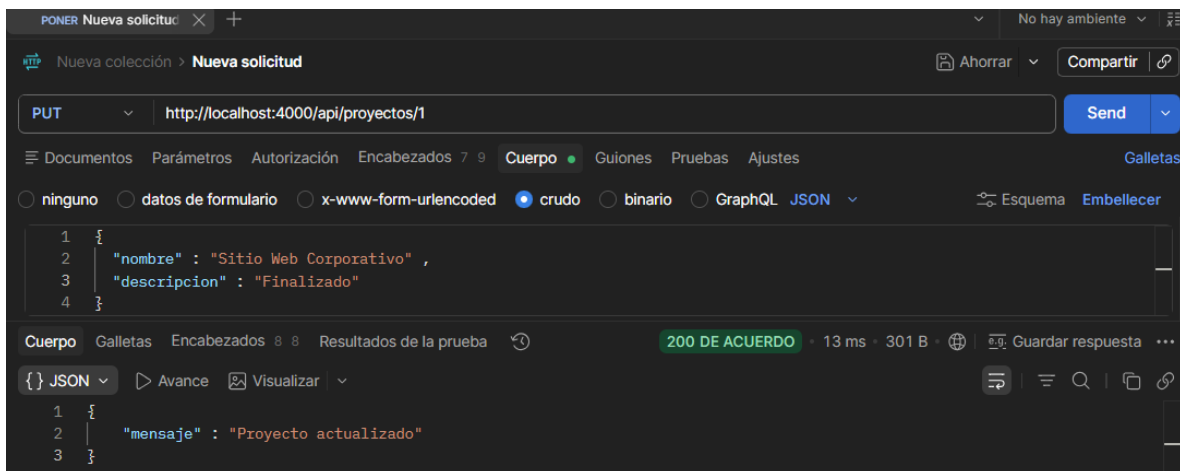
Se realizó una consulta de proyectos mediante una petición GET. El sistema respondió con la información registrada en la base de datos, confirmando la correcta operación del módulo Proyectos.

Prueba 2. Registro de proyecto (POST): URL <http://localhost:4000/api/proyectos>



Se ejecutó una prueba de creación de proyectos utilizando una petición POST. La información fue almacenada correctamente en la base de datos y el sistema confirmó el registro exitoso del nuevo proyecto.

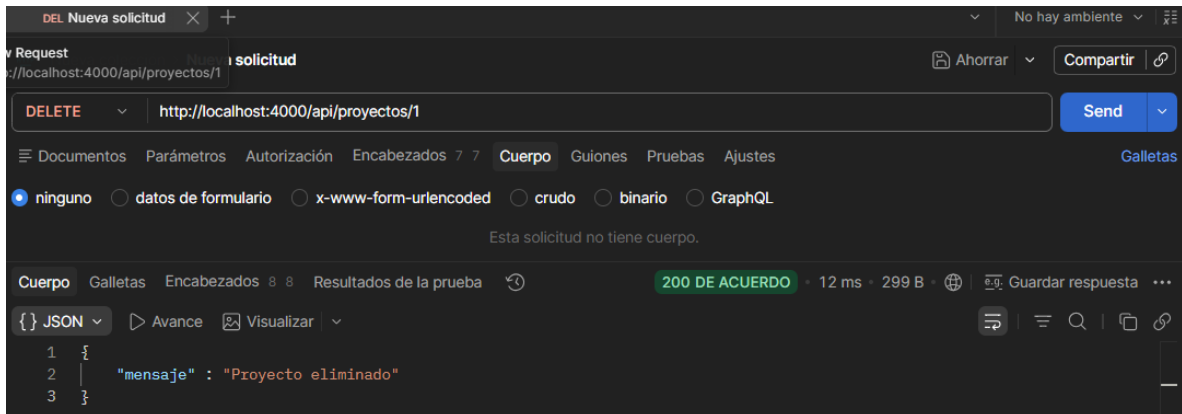
Prueba 3. Actualización de proyecto (PUT): URL <http://localhost:4000/api/proyectos/1>



Se realizó una prueba de actualización de proyectos mediante una petición PUT. El sistema modificó correctamente el estado del proyecto seleccionado y actualizó la información en la base de datos.

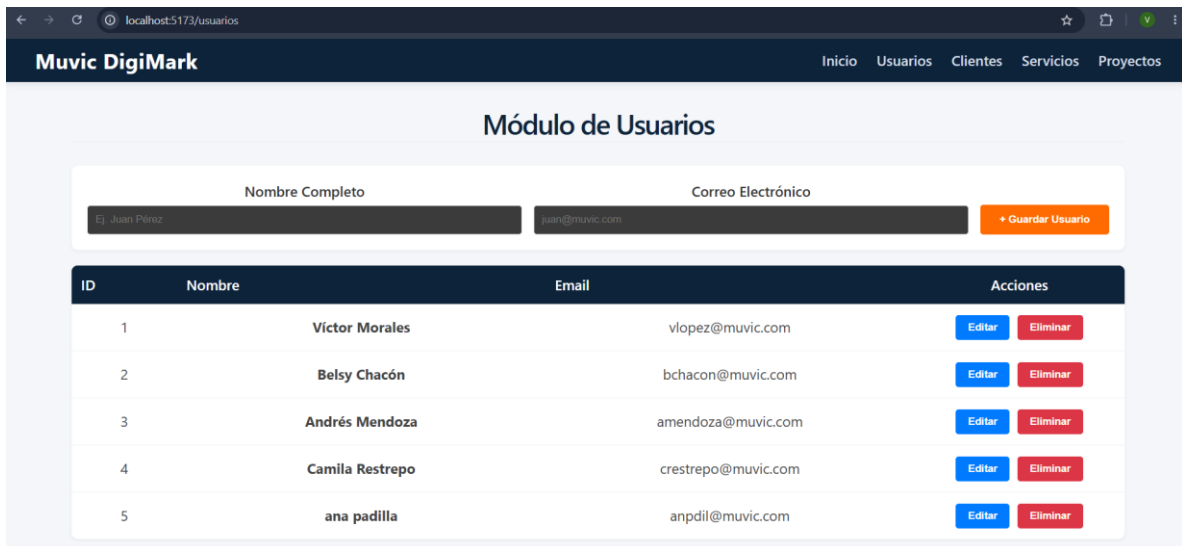
Prueba 4. Eliminación de proyecto (DELETE): URL

<http://localhost:4000/api/proyectos/1>



Se ejecutó una prueba de eliminación utilizando una petición DELETE. El sistema eliminó correctamente el proyecto seleccionado, confirmando el correcto funcionamiento de las operaciones CRUD implementadas.

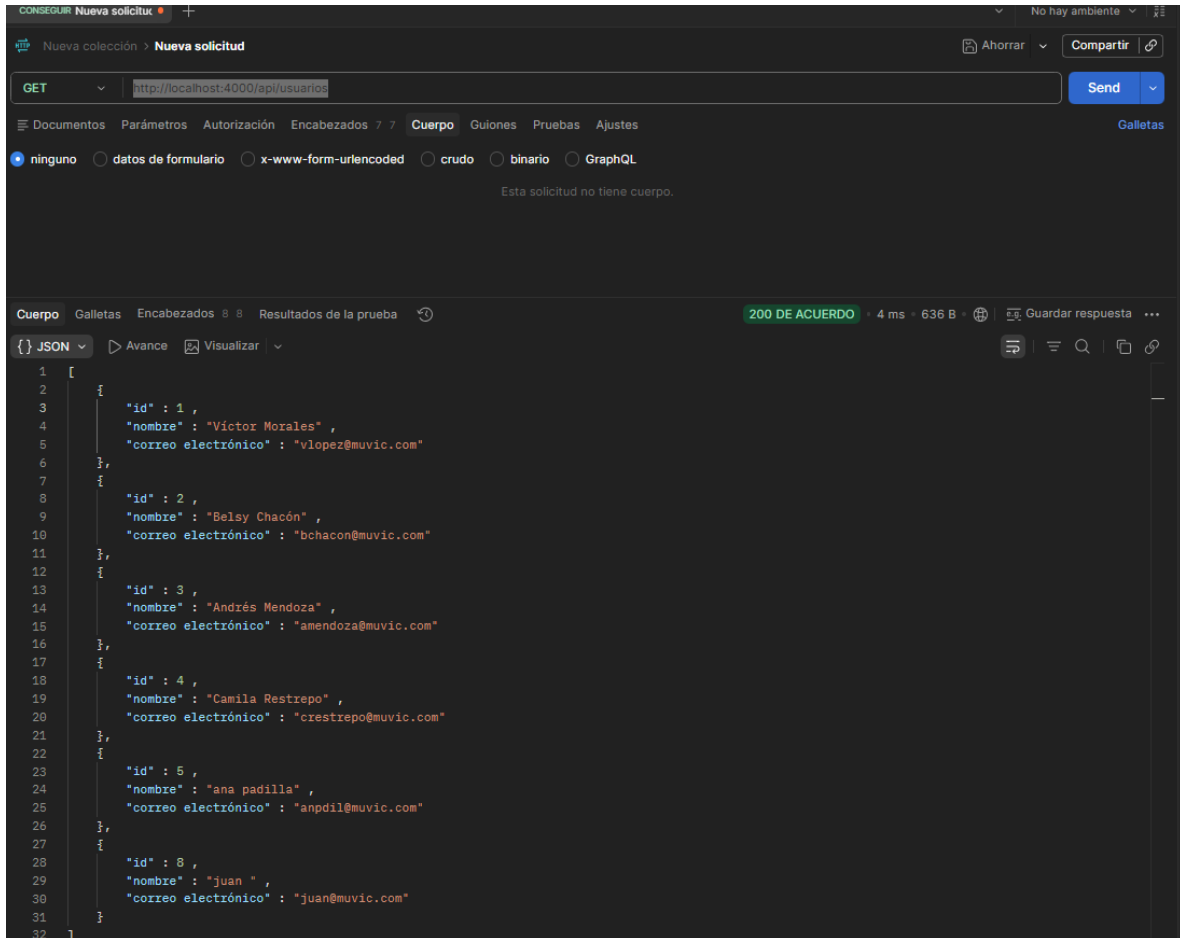
MÓDULO USUARIOS



Permite registrar, consultar, actualizar y eliminar la información de los usuarios del sistema, facilitando la administración de accesos y datos personales.

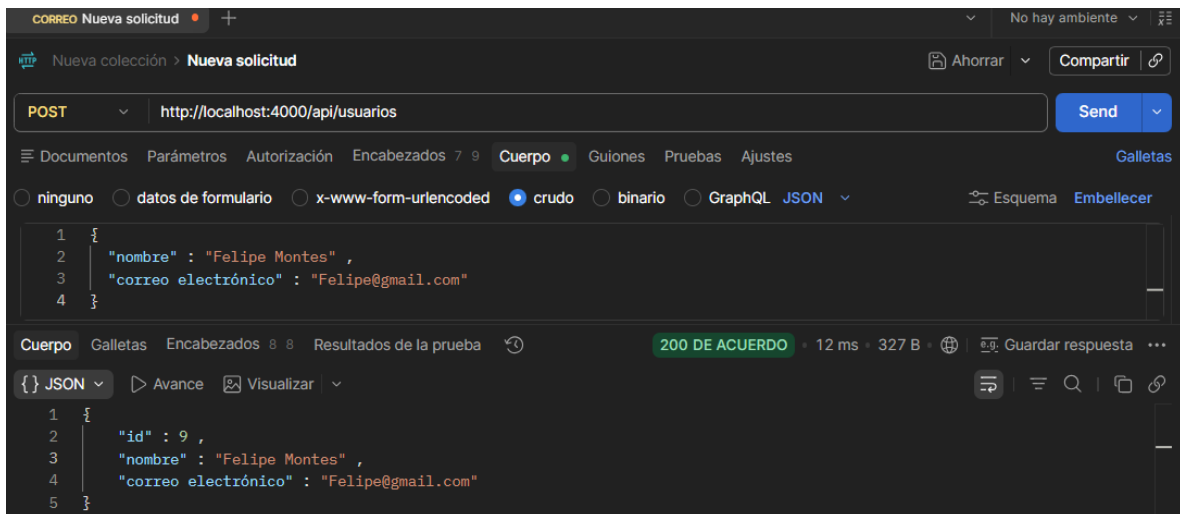
Pruebas al módulo usuarios con postman

Prueba 1. Consulta de usuarios (GET): URL <http://localhost:4000/api/usuarios>



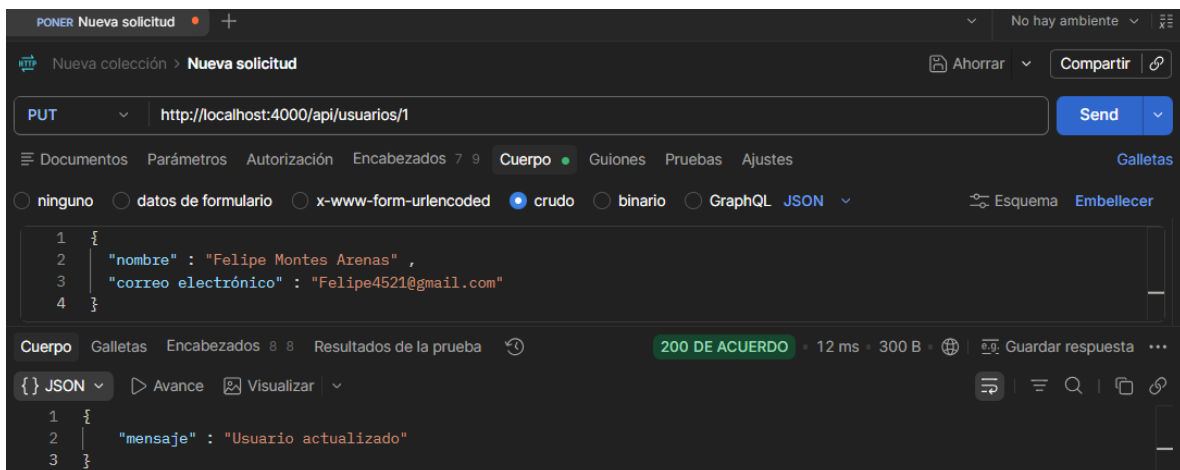
Se realizó una prueba de consulta de usuarios mediante una petición GET al endpoint correspondiente. El sistema respondió con código de estado 200 OK y mostró la información almacenada en la base de datos en formato JSON. Esta prueba permitió verificar la correcta comunicación entre la API REST y la base de datos MySQL.

Prueba 2. Registro de usuarios (POST): URL <http://localhost:4000/api/usuarios>



Se ejecutó una prueba de creación de un usuario utilizando una petición POST. La información fue almacenada correctamente en la base de datos y el sistema confirmó el registro exitoso el nuevo usuario.

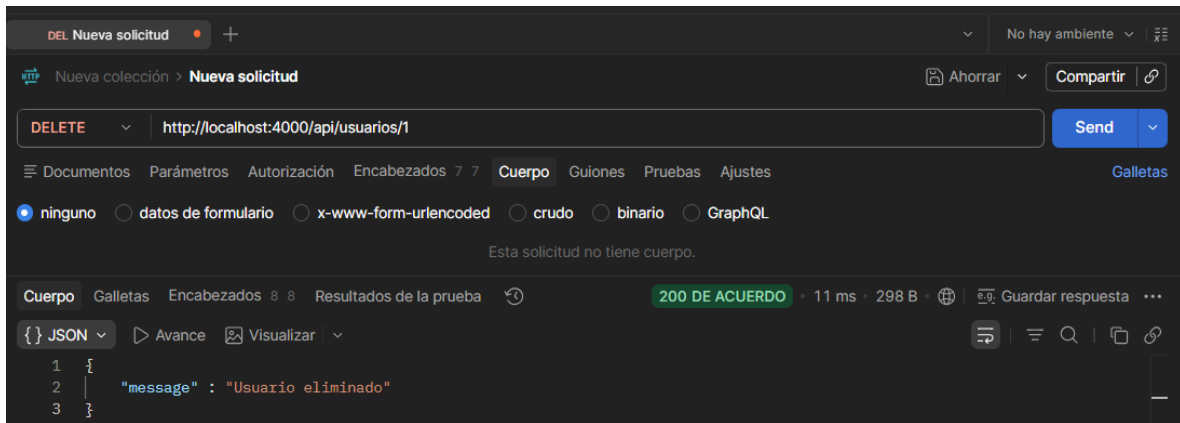
Prueba 3. Actualización de usuarios (PUT): URL <http://localhost:4000/api/usuarios/1>



Se ejecutó una prueba de actualización de información mediante una petición PUT. El sistema modificó correctamente los datos del usuario seleccionado y actualizó el registro correspondiente en la base de datos.

Prueba 4. Eliminación de usuarios (DELETE): URL

<http://localhost:4000/api/usuarios/1>



Se realizó una prueba de eliminación mediante una petición DELETE. El sistema eliminó correctamente a el usuario seleccionado y confirmó la operación realizada sobre la base de datos.

CONCLUSIÓN

En esta evidencia EV02 se logró el desarrollo e integración del módulo de clientes dentro del sistema Muvic DigiMark, aplicando correctamente la creación y consumo de servicios API para la gestión de información. A través de las pruebas realizadas en Postman, se validaron las operaciones CRUD (crear, consultar, actualizar y eliminar), verificando el correcto funcionamiento de cada endpoint y el manejo adecuado de respuestas tanto exitosas como de error.

Además, se fortaleció la comprensión sobre la estructura de una API REST, la comunicación entre frontend y backend, y la importancia de realizar pruebas funcionales para garantizar la calidad del software. Con esto se confirma que el módulo cumple con los requerimientos establecidos y responde de manera adecuada a las solicitudes del sistema.